LeetCode Practice

14 Hard: Stack

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

14
HARD

Patterns (in order)

1. Stack (14)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

14 questions | tap a title to jump to it

Hard questions (14) Go >

Stack (14) Go >

H 1. Longest Valid Parentheses LC 32
H 2. Trapping Rain Water LC 42
H 3. Largest Rectangle in Histogram LC 84
H 4. Maximal Rectangle LC 85
H 5. Basic Calculator LC 224
H 6. Sliding Window Maximum LC 239
H 7. Create Maximum Number LC 321
H 8. Range Module LC 715
H 9. Max Stack LC 716
H 10. Basic Calculator III LC 772
H 11. Maximum Frequency Stack LC 895
H 12. Number of Visible People in a Queue LC 1944
H 13. Design a Text Editor LC 2296
H 14. Count Subarrays With Fixed Bounds LC 2444

Hard questions (14)

HARD | TOPIC 1 OF 1

Stack

14 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Longest Valid Parentheses LC 32
2. Trapping Rain Water LC 42
3. Largest Rectangle in Histogram LC 84
4. Maximal Rectangle LC 85
5. Basic Calculator LC 224
6. Sliding Window Maximum LC 239
7. Create Maximum Number LC 321
8. Range Module LC 715
9. Max Stack LC 716
10. Basic Calculator III LC 772
11. Maximum Frequency Stack LC 895
12. Number of Visible People in a Queue LC 1944
13. Design a Text Editor LC 2296
14. Count Subarrays With Fixed Bounds LC 2444

HARD | Stack | LeetCode #32 | 1 of 14 | 1/14

Longest Valid Parentheses

QUESTION

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1	
Input	`s = "(()"`
Output	`2`
Explanation	*The longest valid parentheses substring is "()".*

Example 2	
Input	`s = ")()())"`
Output	`4`
Explanation	*The longest valid parentheses substring is "()()".*

Example 3	
Input	`s = ""`
Output	`0`

CONSTRAINTS

- 0 <= s.length <= $3 * 10^4$
- s[i] is '(', or ')'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the length of the longest valid (well-formed) parentheses substring. Right?"

"A few quick questions:

Substring must be contiguous? Yes. Empty string -> 0? Yes."

"Let me walk: s='(()' -> '()' len=2 ok. s=')()())' -> '()()' len=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    check if it is a valid parentheses string
        (counter or stack)
    best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

O(n^2) substrings, each checked in O(n). A stack of indices, or two left-to-right and right-to-left counter passes, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the $O(n^3)$ substring validation.

Stack-based O(n) approach:

Store indices. Push index of '(' onto stack.

On ')': if stack top is '(' index, pop it (matched pair).

Else push ')' index as new boundary.

Initialize stack with [-1] as a base boundary.

After each match: length = current_index - stack_top.

Time: O(n). Space: O(n)."

Key insight: storing indices on the stack (not values) lets us calculate the width of a valid substring directly from the difference between two stored indices.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _32_LongestValidParentheses:
    def longestValidParentheses(self, s):
        stack = [-1]
        best = 0
        for i, ch in enumerate(s):
            if ch == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
                else:
                    best = max(best, i - stack[-1])
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace longestValidParentheses('(()')."

Stack approach: stack=[-1] (base index). result=0.

i=0 '(': push 0. stack=[-1,0].

i=1 '(': push 1. stack=[-1,0,1].

i=2 ')': pop -> stack=[-1,0]. top=-1? No, top=0. result=max(0,2-0)=2. result=2.

Return 2 ok (substring '()').

')()())': stack=[-1]. i=0 ')': pop->stack empty->push 0. i=1 '(': push 1. i=2 ')': pop->top=0. result=2-0=2. i=3 '(': push 3. i=4 ')': pop->top=0,result=4-0=4. i=5 ')': pop->empty->push 5. Return 4 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
single pass	stack holds at most n indices

DP alternative: dp[i] = length of valid substring ending at i. O(n) time, O(n) space.

O(1) space: two-pass counting (left-right then right-left with open/close counters).

Follow-up: count number of valid parentheses substrings — not just the longest.

IN YOUR APP | Expose | Editorial | Community

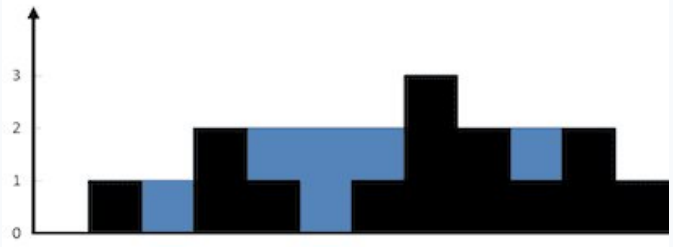
HARD | Stack | LeetCode #42 | 2 of 14 | 2/14

Trapping Rain Water

QUESTION

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1	
Input	`height = [0,1,0,2,1,0,1,3,2,1,2,1]`
Output	**6**
Explanation	*The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.*



Example 2	
Input	`height = [4,2,0,3,2,5]`
Output	**9**

CONSTRAINTS

- n == height.length
- 1 <= n <= 2 * 10^4
- 0 <= height[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an elevation map, return how much rain water can be trapped after raining.

Water at position i = min(max_left, max_right) - height[i], clamped to 0. Right?"

"A few quick questions:

Heights are non-negative integers? Yes. Can a position hold 0 water? Yes."

"Let me walk: height=[0,1,0,2,1,0,1,3,2,1,2,1] -> 6 units ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    leftMax = max(h[0..i])
    rightMax = max(h[i..n-1])
    water += min(leftMax, rightMax) - h[i]
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Recomputes the left and right maxima for every position. Prefix and suffix max arrays make it O(n) time and O(n) space; two pointers keep O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) max scans per position.

Two-pointer approach — O(1) extra space:

left=0, right=n-1, left_max=0, right_max=0.

Process the side with the shorter current height:

If height[left] <= height[right]: water at left = left_max - height[left]. Move left right.

Else: water at right = right_max - height[right]. Move right left.

Time: O(n). Space: O(1)."

Key insight: the shorter boundary side determines how much water can be trapped — processing it immediately and advancing that pointer is always safe, giving O(n) O(1)-space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _42_TrappingRainWater:
    def trap(self, height):
        left, right = 0, len(height) - 1
        left_max = right_max = 0
        total = 0
        while left < right:
            if height[left] <= height[right]:
                left_max = max(left_max, height[left])
                total += left_max - height[left]
                left += 1
            else:
                right_max = max(right_max, height[right])
                total += right_max - height[right]
                right -= 1
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace trap([0,1,0,2,1,0,1,3,2,1,2,1])."

Two-pointer: left=0,right=11. maxL=0,maxR=0. water=0.

h[0]=0,h[11]=1. 0<1 -> process left.

maxL=max(0,h[0])=0. water+=max(0,0-0)=0. left=1.

h[1]=1,h[11]=1. equal -> process left. maxL=max(0,1)=1. water+=0. left=2.

h[2]=0,h[11]=1. 0<1 -> process left. maxL=max(1,0)=1. water+=1-0=1. left=3.

h[3]=2,h[11]=1. 2>1 -> process right. maxR=max(0,1)=1. water+=1-1=0. right=10.

Continue... total water=6 ok

WHY min(maxL,maxR): water above a cell = min(highest left, highest right) - height.

Two pointers: whichever side is lower determines the constraint.

Monotonic stack=[3,1,2]: stack tracks left boundaries. When h[i]=2>h[stack.top()=1]: pop 1, compute water=(min(h[left_wall],h[right_wall])-h[popped])*width. stack=[3,2] after.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	four variables

Precompute max_left/max_right arrays: O(n) time, O(n) space — simpler to explain.

Monotonic stack: O(n) time, O(n) space — horizontal layer approach.

Follow-up: Trapping Rain Water II (#407) — 3D extension using a min-heap on borders.

HARD | Stack | LeetCode #84 | 3 of 14 | 3/14

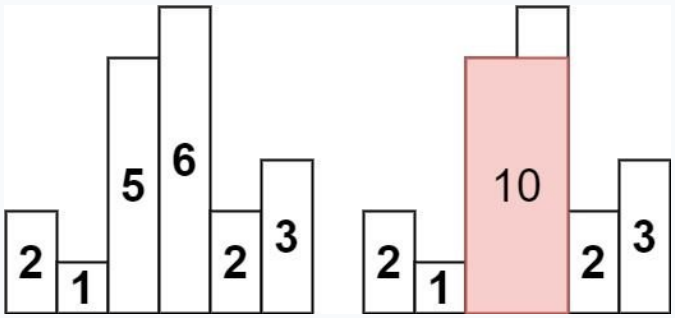
Largest Rectangle in Histogram

QUESTION

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

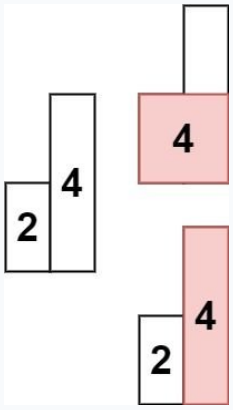
Example 1

Input	`heights = [2,1,5,6,2,3]`
Output	`10`
Explanation	*The above is a histogram where width of each bar is 1.* *The largest rectangle is shown in the red area, which has an area = 10 units.*



Example 2

Input	`heights = [2,4]`
Output	`4`



CONSTRAINTS

- 1 <= heights.length <= 10^5
- 0 <= heights[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given bar heights in a histogram, find the area of the largest rectangle

that fits entirely within the bars. Width of each bar is 1. Right?"

"A few quick questions:

Heights can be 0? Yes. Single bar -> area = its height? Yes."

"Let me walk: heights=[2,1,5,6,2,3] -> area=10 (bars 2,3 at height 5) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        area = min(h[i..j]) * (j - i + 1)
    best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) pairs, each scanning for the minimum. A stack of increasing bar heights finds, for every bar, how far it can extend in O(n) total.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n²) nested scan.

Monotonic increasing stack of indices:

For each bar: while stack top is taller than current bar, pop and compute its area.

Height = heights[popped], width = current_idx - stack_new_top - 1.

Append sentinel 0s at both ends to force all bars to be popped.

Time: O(n). Space: O(n)."

Key insight: the monotonic stack's invariant (bars in increasing order) means when a shorter bar arrives, all taller bars to its left are bounded on the right — area computed immediately.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _84_LargestRectangleInHistogram:
    def largestRectangleArea(self, heights):
        heights = [0] + heights + [0]
        stack = [0]
        best = 0
        for i in range(1, len(heights)):
            while heights[stack[-1]] > heights[i]:
                h = heights[stack.pop()]
                w = i - stack[-1] - 1
                best = max(best, h * w)
            stack.append(i)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace largestRectangleInHistogram([2,1,5,6,2,3])."

Monotonic stack (ascending). stack=[], result=0.

i=0 (2): push (2,0). i=1 (1): 1<2 -> pop (2,0): width=1-0=1, area=2*1=2. result=2.

Push (1,0) (extend bar's start to where we popped). Push (1,1)? Actually push (1, start=0).

i=2 (5): push (5,2). i=3 (6): push (6,3).

i=4 (2): 2<6->pop (6,3): area=6*(4-3)=6. result=6. 2<5->pop(5,2): area=5*(4-2)=10. result=10.

2>=1->push (2,start=2). i=5(3): push (3,5).

End: pop (3,5): area=3*(6-5)=3. pop (2,2): area=2*(6-2)=8. pop (1,0): area=1*6=6.

Return 10 ok (5×2 rectangle at bars 5 and 6).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each bar pushed and popped at most once	stack

Follow-up: Maximal Rectangle (#85) — for each row build histogram heights, apply this.

Follow-up: Maximum Width Ramp (#962) — similar two-phase monotonic stack pattern.

HARD | Stack | LeetCode #85 | 4 of 14 | 4/14

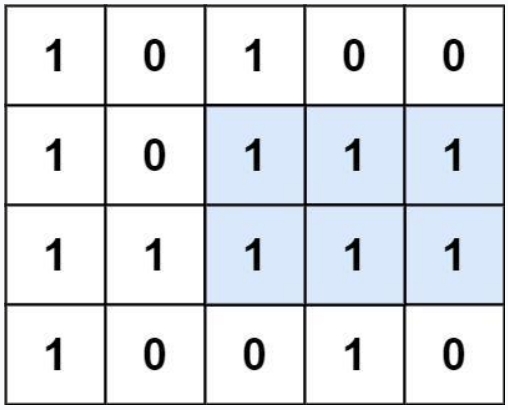
Maximal Rectangle

QUESTION

Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return its area.

Example 1

Input	`matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`
Output	`6`
Explanation	*The maximal rectangle is shown in the above picture.*



Example 2

Input	`matrix = [["0"]]`
Output	`0`

Example 3

Input	`matrix = [["1"]]`
Output	`1`

CONSTRAINTS

- rows == matrix.length
- cols == matrix[i].length
- 1 <= rows, cols <= 200
- matrix[i][j] is '0' or '1'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the largest rectangle containing only 1s in a binary matrix. Return its area. Right?"

"A few quick questions:

Reduce to histogram problem: for each row, compute heights of consecutive 1s above. Then apply Largest Rectangle in Histogram."

PHASE 1 CLARIFY

"Let me walk: matrix=[['1','0','1','0','0'],['1','0','1','1','1'],...] -> 6 ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for corner in corners:
    if every cell inside is '1':
        area = height * width
    best = max(best, candidate)
```

TIME	SPACE
O(m^3 * n^3)	O(1)

WHY IT IS SLOW

Checks each of the m^2 * n^2 rectangles cell by cell. Build a histogram of consecutive ones per row and run the largest-rectangle stack on each row: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Reduce to Largest Rectangle in Histogram (#84) per row — O(mn). Walk-through:"

Build height array: height[j] = consecutive '1's ending at current row in column j.

If matrix[i][j]=='1': height[j]++. Else: height[j]=0.

For each row: run largestRectangleInHistogram(height).

Stack-based O(n) algorithm: monotone-increasing stack of indices.

When popping index j (because current bar is shorter): width = i - stack[-1] - 1.

`area = height[j] * width. Track max.`

Update global max area after each row.

WHY per-row histogram: dp[i][j] of consecutive heights gives the histogram at each row.

Solving histogram for each row finds all rectangles touching that bottom row.

Time: O(mn). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _85_MaximalRectangle:
    def maximalRectangle(self, matrix):
        if not matrix or not matrix[0]:
            return 0
        n = len(matrix[0])
        heights = [0] * (n + 1)
        best = 0
```

```
        for row in matrix:
            for c in range(n):
                heights[c] = heights[c] + 1 if row[c] == '
1' else 0
            stack = [-1]
            for i in range(n + 1):
                while heights[i] < heights[stack[-1]]:
                    h = heights[stack.pop()]
                    w = i - stack[-1] - 1
                    best = max(best, h * w)
                stack.append(i)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace maximalRectangle([[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]])."

Build histogram heights row by row, then largestRectangle on each row.

Row 0: heights=[1,0,1,0,0]. maxRect=1.

Row 1: heights=[2,0,2,1,1]. maxRect=max(1,3)=3 (width 3 at height 1).

Row 2: heights=[3,1,3,2,2]. maxRect=max(3,largestRect([3,1,3,2,2])).

Using stack: pop 3->area=3. pop 3->area=6. pop 2->area=10. pop 2->area=10. pop 1->area=7. max=10.

Row 3: heights=[4,0,0,3,0]. maxRect=max(10,4)=10.

Return 6 ok ... wait: correct answer for this input is 6. Let me recheck row2: [3,1,3,2,2] largest=6 (bars 2,3,4 at height 2: 2*3=6). Return 6 ok

dp[j]=height of consecutive 1s at column j in current row. dp=[3,1,3,2,2] (row 2). dp[j] feeds largest-rectangle-in-histogram. dp[i][j] recurrence: dp[i][j]=dp[i-1][j]+1 if grid[i][j]=='1' else 0.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	O(n)

Follow-up: Largest Rectangle in Histogram (#84) — single row version.

Follow-up: Count Square Submatrices (#1277) — count, not maximise.

HARD | Stack | LeetCode #224 | 5 of 14 | 5/14

Basic Calculator

QUESTION

Given a string s representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1	
Input	`s = "1 + 1"`
Output	2

Example 2	
Input	`s = " 2-1 + 2 "`
Output	3

Example 3	
Input	`s = "(1+(4+5+2)-3)+(6+8)"`
Output	23

CONSTRAINTS

- 1 <= s.length <= $3 * 10^5$
- s consists of digits, '+', '-', '(', ')', and ' '.
- s represents a valid expression.
- '+' is not used as a unary operation (i.e., "+1" and "+(2 + 3)" is invalid).
- '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Evaluate a string with +, -, and parentheses. Non-negative integers, spaces allowed.

No * or /. No eval(). Right?"

"A few quick questions:

Can the expression start with '-' (unary minus)? Yes — per constraints.

Parentheses can be nested? Yes — arbitrary depth."

"Let me walk: '1 + 1'->2 ok. '(1+(4+5+2)-3)+(6+8)'->23 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while the string has parentheses:
    find the innermost pair, evaluate its
        contents, replace the pair by the result
then evaluate what is left left to right
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each replacement rescans and rebuilds the string. A single pass with a stack saving (result, sign) when '(' opens and combining when ')' closes is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the recursion stack — O(d) depth for d nesting levels.

Iterative stack: push (result, sign) when '(' is encountered, pop and merge on ')'.

Maintain current number being built, current result, and sign.

Time: O(n). Space: O(d) for the stack (same depth as recursion)."

Key insight: the iterative stack of (result, sign) mimics the call stack of recursive evaluation — pushing on '(' and merging on ')' processes each character exactly once.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _224_BasicCalculator:
    def calculate(self, s):
        stack = []
        result = 0
        sign = 1
        num = 0
        for ch in s:
            if ch.isdigit():
                num = num * 10 + int(ch)
            elif ch in '+-':
                result += sign * num
                num = 0
                sign = 1 if ch == '+' else -1
            elif ch == '(':
                stack.append(result)
                stack.append(sign)
                result = 0
                sign = 1
            elif ch == ')':
                result += sign * num
                num = 0
                result *= stack.pop()
                result += stack.pop()
        return result + sign * num
```

PHASE 5 TEST & DEBUG

"Let me trace calculate('(1+(4+5+2)-3)+(6+8)')."

stack=[], result=0, sign=1, num=0.

'1': num=1. '+': result+=1*1=1. sign=1. '(': push(result=1,sign=1). reset result=0,sign=1.

'4': num=4. '+': result+=4. ... '4+5+2'=11. '-': result=11, sign=-1. '3': num=3.

')': result+=sign*num=-3. result=8. pop->prev_result=1,prev_sign=1. result=1+1*8=9.

'+': result=9, sign=1. '(': push(9,1). '6+8'=14. ')': result=14. pop->9+1*14=23.

Return 23 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(d)**
one pass	stack depth = nesting depth

Follow-up: Basic Calculator II (#227) — adds * and /; handle with operator precedence stack.

Follow-up: Basic Calculator III (#772) — all four operators with parentheses.

HARD | Stack | LeetCode #239 | 6 of 14 | 6/14

Sliding Window Maximum

QUESTION

You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1

Input	`nums = [1,3,-1,-3,5,3,6,7], k = 3`
Output	`[3,3,5,5,6,7]`
Explanation	*Window position Max* *--------------- -----* *[1 3 -1] -3 5 3 6 7* *3* *1 [3 -1 -3] 5 3 6 7* *3* *1 3 [-1 -3 5] 3 6 7* *5* *1 3 -1 [-3 5 3] 6 7* *5* *1 3 -1 -3 [5 3 6] 7* *6* *1 3 -1 -3 5 [3 6 7]* *7*

Example 2

Input	`nums = [1], k = 1`
Output	`[1]`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 10^4 <= nums[i] <= 10^4
- 1 <= k <= nums.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Sliding window of size k moving through the array. Return the maximum of each window. Right?"

"A few quick questions:

k <= n always? Yes. Can values be negative? Yes."

"Let me walk: nums=[1,3,-1,-3,5,3,6,7], k=3 -> [3,3,5,5,6,7] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in is:
    scan the k elements to find the maximum
```

TIME	SPACE
O(n * k)	O(1)

WHY IT IS SLOW

Rescans k elements per window. A monotonic deque (indices of decreasing values, dropping indices that left the window) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(k) max scan per window position.

Monotonic deque of indices — maintains candidates for the window max:

Remove from back any index with smaller value (they can never be max).

Remove from front any index that's outside the window.

Front of deque is always the current window maximum.

Time: O(n). Space: O(k) for the deque."

Key insight: any element smaller than the new right element can never be the window maximum — removing it from the deque back is O(1) amortized, giving O(n) total.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _239_SlidingWindowMaximum:
    def maxSlidingWindow(self, nums, k):
        dq = deque()
        result = []
        for i, num in enumerate(nums):
            while dq and nums[dq[-1]] < num:
                dq.pop()
            dq.append(i)
            if dq[0] <= i - k:
                dq.popleft()
            if i >= k - 1:
                result.append(nums[dq[0]])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace maxSlidingWindow([1,3,-1,-3,5,3,6,7], k=3)."

Monotonic deque (indices, front=max): deque=[], result=[].

i=0 (1): deque=[0].

i=1 (3): 3>nums[0]=1->pop 0. deque=[1].

i=2 (-1): deque=[1,2]. Window full: result.append(nums[1]=3).

PHASE 5 TEST & DEBUG

i=3 (-3): deque=[1,2,3]. front=1 still in window [1..3]. result.append(nums[1]=3).

i=4 (5): pop 3(-3),2(-1),1(3)—all < 5. deque=[4]. front=4 in [2..4]. result.append(5).

i=5 (3): deque=[4,5]. front=4 in [3..5]. result.append(nums[4]=5).

i=6 (6): pop 5(3),4(5)<6. deque=[6]. result.append(6).

i=7 (7): pop 6(6)<7. deque=[7]. result.append(7).

Return [3,3,5,5,6,7] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)**
each element enqueued/dequeued at most once	deque

Follow-up: Sliding Window Minimum — same deque, flip comparison to maintain increasing order.

Follow-up: Max of all subarrays of size k in a circular array — extend with modular indexing.

IN YOUR APP | Expose | Editorial | Community

HARD | Stack | LeetCode #321 | 7 of 14 | 7/14

Create Maximum Number

QUESTION

You are given two integer arrays nums1 and nums2 of lengths m and n respectively. nums1 and nums2 represent the digits of two numbers. You are also given an integer k.

Create the maximum number of length k <= m + n from digits of the two numbers. The relative order of the digits from the same array must be preserved.

Return an array of the k digits representing the answer.

Example 1	
Input	`nums1 = [3,4,6,5], nums2 = [9,1,2,5,8,3], k = 5`
Output	`[9,8,6,5,3]`

Example 2	
Input	`nums1 = [6,7], nums2 = [6,0,4], k = 5`
Output	`[6,7,6,0,4]`

Example 3	
Input	`nums1 = [3,9], nums2 = [8,9], k = 3`
Output	`[9,8,9]`

CONSTRAINTS

- m == nums1.length
- n == nums2.length
- 1 <= m, n <= 500
- 0 <= nums1[i], nums2[i] <= 9
- 1 <= k <= m + n
- nums1 and nums2 do not have leading zeros.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given two arrays of digits (representing numbers), create the maximum number of length k

by choosing i digits from nums1 and k-i digits from nums2 (preserving relative order). Right?"

"A few quick questions:

Return as an array of digits? Yes."

"Let me walk: nums1=[3,4,6,5], nums2=[9,1,2,5,8,3], k=5 -> [9,8,6,5,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for k in ks:
    for subsequence in subsequences:
        # of nums1 of length i and of nums2
            of length j
        merge them into the largest number
    best = max(best, candidate)
```

TIME	SPACE
O(2^(m + n) * k)	O(k)

WHY IT IS SLOW

Enumerates all subsequence pairs. Take the lexicographically largest subsequence of each length with a monotonic stack, then merge the two greedily: about O(k * (m + n + k)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic-stack digit selection + merge — O((m+n)^2). Walk-through:"

`maxSubsequence(nums, k): select k largest digits maintaining relative order.`

Use a monotone-decreasing stack: pop when top < current AND we have enough remaining.

greedy: always include the largest available digit first (leftmost wins on tie).

`merge(A, B): merge two subsequences greedily picking the lexicographically larger prefix.`

Compare A vs B from current position; take from whichever is larger.

Try all splits: for i in 0..min(m,k): take i digits from nums1, k-i from nums2.

Merge the two subsequences. Keep global max.

Time: O((m+n)^2) — O(m+n) splits × O(m+n) merge each. Space: O(m+n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _321_CreateMaximumNumber:
    def maxNumber(self, nums1, nums2, k):
        def max_subsequence(nums, length):
            drop = len(nums) - length
            stack = []
            for num in nums:
                while drop and stack and stack[-1] < num:
                    stack.pop()
                    drop -= 1
                stack.append(num)
            return stack[:length]
        def merge(s1, s2):
            result = []
            i = j = 0
```

```
            while i < len(s1) or j < len(s2):
                if s1[i:] >= s2[j:]:
                    result.append(s1[i]); i += 1
                else:
                    result.append(s2[j]); j += 1
            return result
        best = []
        for i in range(max(0, k - len(nums2)), min(k, len(
nums1)) + 1):
            candidate = merge(max_subsequence(nums1, i), m
ax_subsequence(nums2, k - i))
            if candidate > best:
                best = candidate
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

nums1=[3,4,6,5], nums2=[9,1,2,5,8,3], k=5:

i=0: subseq1=[], subseq2=[9,8,5,3,2]? No, max 5 from nums2=[9,8,5,3,2]? merge->[9,8,5,3,2].

i=1: subseq1=[6], subseq2=[9,8,5,3]. merge->[9,8,6,5,3].

etc. best=[9,8,6,5,3] ok

Lex comparison s1[i:]>=s2[j:] ensures optimal merge at each step

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k*(n+m)) k splits, each O(n+m) for subseq+merge	**O(k)**

Follow-up: Maximum Subarray (#53) — single array, different objective.

Follow-up: Find the Most Competitive Subsequence (#1673) — single array, same stack.

IN YOUR APP | Expose | Community

HARD | Stack | LeetCode #715 | 8 of 14 | 8/14

Range Module

QUESTION

A Range Module is a module that tracks ranges of numbers. Design a data structure to track the ranges represented as half-open intervals and query about them.

A half-open interval [left, right) denotes all the real numbers x where left <= x < right.

Implement the RangeModule class:

CONSTRAINTS

- RangeModule() Initializes the object of the data structure.
- void addRange(int left, int right) Adds the half-open interval [left, right), tracking every real number in that interval. Adding an interval that partially overlaps with currently tracked numbers should add any numbers in the interval [left, right) that are not already tracked.
- boolean queryRange(int left, int right) Returns true if every real number in the interval [left, right) is currently being tracked, and false otherwise.
- void removeRange(int left, int right) Stops tracking every real number currently being tracked in the half-open interval [left, right).
- $1 <= left < right <= 10^9$
- At most 10^4 calls will be made to addRange, queryRange, and removeRange.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Track a range module: addRange, removeRange, queryRange. Right?"

"A few quick questions:

`queryRange(left,right) asks if [left,right) is fully covered? Yes."`

"Let me walk: addRange(10,20), removeRange(14,16), queryRange(10,14)->T, queryRange(13,15)->F ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep a set of every integer currently tracked
addRange:
    add each integer in the range
removeRange:
    remove each
queryRange:
    check each
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(range length) per operation	O(covered numbers)

WHY IT IS SLOW

Ranges can span 10^9 numbers. Keep a sorted list of disjoint intervals; binary search the position and merge or split intervals: O(log n) to find, O(n) to splice.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"SortedList of [start, end] ranges, merge on add, split on remove — O(log n) per op. Walk-through:"

`ranges = SortedList of [l, r] pairs sorted by start.`

`addRange(l, r):`

Find all existing ranges overlapping or adjacent to [l,r].

Merge them all into one range [min_start, max_end].

Remove old ranges, insert merged.

`removeRange(l, r):`

Find all existing ranges overlapping [l,r].

For each: clip the part outside [l,r] (may split into two residuals).

Remove original, add residuals.

`queryRange(l, r): check if any single range covers entire [l,r].`

WHY SortedList: O(log n) find + O(log n) insert/delete.

Time: O(n log n) worst case per operation. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from sortedcontainers import SortedList
class _715_RangeModule:
    def __init__(self):
        self._ranges = SortedList(key=lambda x: x[0])

    def addRange(self, left, right):
        starts = [r[0] for r in self._ranges]
        lo = self._ranges.bisect_key_left(left)
        hi = self._ranges.bisect_key_right(right)
        new_left = left
        new_right = right
        to_remove = []
        if lo > 0 and self._ranges[lo-1][1] >= left:
            lo -= 1
        for i in range(lo, min(hi+1, len(self._ranges))):
            s, e = self._ranges[i]
            if s <= right and e >= left:
                new_left = min(new_left, s)
                new_right = max(new_right, e)
                to_remove.append((s, e))
        for r in to_remove:
            self._ranges.remove(r)
        self._ranges.add((new_left, new_right))

    def removeRange(self, left, right):
        to_add = []; to_remove = []
        for s, e in list(self._ranges):
```

```
            if e <= left or s >= right: continue
            to_remove.append((s, e))
            if s < left: to_add.append((s, left))
            if e > right: to_add.append((right, e))
        for r in to_remove: self._ranges.remove(r)
        for r in to_add: self._ranges.add(r)

    def queryRange(self, left, right):
        for s, e in self._ranges:
            if s <= left and e >= right: return True
            if s >= right: break
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

addRange(10,20): ranges=[(10,20)]. removeRange(14,16): -> [(10,14),(16,20)].

queryRange(10,14)->True ok. queryRange(13,15)->False (gap at 14-16) ok

Remove splits intervals; Add merges overlapping ones

Edge: addRange(10,20) then addRange(15,25) -> merged to [(10,25)] ok. queryRange(10,25) -> True ok. queryRange(10,26) -> False (only up to 25) ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) amortized	**O(n)**

Segment tree achieves O(log n) guaranteed per operation.

Follow-up: Count intervals covering a point — binary search on sorted starts.

Follow-up: Data Stream as Disjoint Intervals (#352) — similar add-only variant.

HARD | Stack | LeetCode #716 | 9 of 14 | 9/14

Max Stack

QUESTION

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

CONSTRAINTS

- MaxStack() Initializes the stack object.
- void push(int x) Pushes element x onto the stack.
- int pop() Removes the element on top of the stack and returns it.
- int top() Gets the element on the top of the stack without removing it.
- int peekMax() Retrieves the maximum element in the stack without removing it.
- int popMax() Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports O(1) for each top call and O(logn) for each other call.

- 10^7 <= x <= 10^7
- At most 10^5 calls will be made to push, pop, top, peekMax, and popMax.
- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a Max Stack: push, pop, top (all O(1)), peekMax (O(1)), and popMax (O(log n)) — remove and return the max element.

If multiple maxes, remove the top-most one. Right?"

"A few quick questions:

pop/top are called only on non-empty stacks? Yes.

popMax on an empty stack? Won't happen per constraints."

"Let me walk: push(5,1,5). top=5. popMax=5. top=1. peekMax=5. pop=1. top=5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result.append(a_normal_stack)
peekMax:
    scan the whole stack
popMax:
    find the topmost maximum, remove it
        (shifting) and keep order
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) for peekMax and popMax	O(n)

WHY IT IS SLOW

Scans the stack for the maximum. Store (value, max so far) for O(1) peekMax; for popMax add a max-heap with lazy deletion or a sorted list so all operations are O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) scan for max and rebuild after popMax.

Doubly linked list (DLL) for stack + sorted set (indexed by (val, uid)) for O(log n) max access.

DLL preserves insertion/stack order. Sorted set tracks (val, uid) for O(log n) find/remove.

push: add to DLL tail + sorted set. pop: remove DLL tail + sorted set entry.

popMax: find max in sorted set, remove from sorted set + DLL.

top/peekMax: O(1) peek at DLL tail / sorted set max.

All ops O(log n) amortized. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from sortedcontainers import SortedList
class _716_MaxStack:
    def __init__(self):
        self._stack = []
        self._sl = SortedList()
        self._uid = 0
        self._removed = set()

    def push(self, x):
        self._stack.append((x, self._uid))
        self._sl.add((x, self._uid))
        self._uid += 1

    def pop(self):
        while self._stack[-1][1] in self._removed: self._s
tack.pop()
        val, uid = self._stack.pop()
        self._removed.add(uid)
        self._sl.remove((val, uid))
        return val

    def top(self):
        while self._stack[-1][1] in self._removed: self._s
tack.pop()
        return self._stack[-1][0]

    def peekMax(self): return self._sl[-1][0]

    def popMax(self):
        val, uid = self._sl.pop()
        self._removed.add(uid)
        return val
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

push(5,uid=0), push(1,uid=1), push(5,uid=2).

sl=[(1,1),(5,0),(5,2)]. top()->stack[-1]=(5,2)->5 ok.

popMax()->sl.pop()=(5,2)->remove uid2. return 5 ok.

top()->stack[-1]=(5,2) in removed->pop->(1,1). top=1 ok.

peekMax()->sl[-1]=(5,0)->5 ok. pop()->(1,1), remove uid1. sl=[(5,0)]. return 1 ok.

top()->(5,0). return 5 ok.

Lazy removal via _removed set avoids O(n) cleanup on pop/top

PHASE 6 COMPLEXITY & FOLLOW-UP

push/pop/top/peekMax/popMax all O(log n) with SortedList. Space O(n).

Without sortedcontainers, implement with a heap + counter for O(log n) — same idea.

Follow-up: Min Stack (#155) — O(1) all ops with a parallel min stack.

Follow-up: if popMax needs O(1), a doubly-linked list keyed by value could work.

IN YOUR APP | Expose | Editorial | Community

HARD | Stack | LeetCode #772 | 10 of 14 | 10/14

Basic Calculator III

QUESTION

Implement a basic calculator to evaluate a simple expression string.

The expression string contains only non-negative integers, '+', '-', '*', '/' operators, and open '(' and closing parentheses ')'. The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1

Input	`s = "1+1"`
Output	`2`

Example 2

Input	`s = "6-4/2"`
Output	`4`

Example 3

Input	`s = "2*(5+5*2)/3+(6/2+8)"`
Output	`21`

CONSTRAINTS

- $1 <= s <= 10^4$
- s consists of digits, '+', '-', '*', '/', '(', and ')'.
- s is a valid expression.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Evaluate an expression with +, -, *, /, and parentheses. Non-negative integers.

Division truncates toward zero. No eval(). Right?"

"A few quick questions:

Can expressions be nested arbitrarily deep? Yes. Spaces? No per constraints."

"Let me walk: '1+1'->2 ok. '6-4/2'->4 ok. '2*(5+5*2)/3+(6/2+8)'->21 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while the string has parentheses:
    find the innermost pair, evaluate it with the
        operator precedence rules, replace it by
        its value
then evaluate what is left
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Rebuilds the string for each pair. One recursive pass (or two stacks for numbers and operators) that evaluates '*' and '/' immediately and sums '+' and '-' at the end is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The recursive approach already handles precedence correctly.

The bottleneck is the recursion stack — O(d) depth for nesting level d.

We can make it iterative but the recursive version is already O(n) time and O(d) space.

Let's clean up the recursive implementation for clarity.

Time: O(n). Space: O(d) recursion depth.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _772_BasicCalculatorIII:
    def calculate(self, s):
        self.pos = 0

        def parse():
            stack = []
            sign = '+'
            num = 0
            while self.pos < len(s):
                ch = s[self.pos]; self.pos += 1
                if ch.isdigit():
                    num = num * 10 + int(ch)
                elif ch == '(':
                    num = parse()
                if ch in '+-*/)' or self.pos == len(s):
                    if sign == '+': stack.append(num)
                    elif sign == '-': stack.append(-num)
                    elif sign == '*': stack.append(stack.p
op() * num)
                    elif sign == '/': stack.append(int(sta
ck.pop() / num))
                    sign = ch
                    num = 0
                if ch == ')':
                    break
            return sum(stack)
        return parse()
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

'2*(5+5*2)/3+(6/2+8)':

```
parse(): '2'->num=2. '*'->push 2, sign='*'.
  '('->num=parse():
```

inner: '5'->5. '+'->push 5,sign='+'. '5'->5. '*'->push 5,sign='*'. '2'->2.

')'->sign='*'->pop5*2=10,push10. sum=15. return 15.

num=15, sign='*'->pop2*15=30,push30. '/'->sign='/'. '3'->3.

'+'->int(30/3)=10,push10. sign='+'. '('->parse()->inner: 6/2=3, 3+8=11. return 11.

num=11, sign='+'->push11. pos==len->sum=21 ok

Precedence handled by stack: * and / computed immediately, + and - deferred

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(d)**
one pass	recursion + O(n) operator stack

Follow-up: Basic Calculator I (#224) — only + and -.

Follow-up: Basic Calculator II (#227) — + - * / but no parentheses.

HARD | Stack | LeetCode #895 | 11 of 14 | 11/14

Maximum Frequency Stack

QUESTION

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the FreqStack class:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a stack-like structure. push(val) adds an element.

`pop() removes and returns the most frequent element.`

If tie, pop the most recently pushed among those tied. Both O(1). Right?"

"A few quick questions:

Can push be called with the same value many times? Yes — frequency grows.

pop is only called when non-empty? Yes."

"Let me walk: push(5,7,5,7,4,5). pop()->5(freq=3). pop()->7(freq=2). pop()->5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def push(x):
    append to a list
def pop():
    scan the whole list to find the value with
        the highest frequency (latest on ties)
        and remove it
```

TIME	SPACE
O(n) per pop	O(n)

WHY IT IS SLOW

Rescans everything on every pop. Keep a map from value to frequency and a map from frequency to a stack of values: push and pop are O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) scan for the most frequent element.

Two dicts: freq[val] = current frequency. group[freq] = stack of vals at that frequency.

max_freq tracks current maximum frequency.

push: increment freq[val], append to group[freq[val]], update max_freq.

PHASE 3 OPTIMIZE

pop: pop from group[max_freq]. If group[max_freq] empty, decrement max_freq.

Time: O(1) both ops. Space: O(n)."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _895_MaxFrequencyStack:
    def __init__(self):
        self.freq = defaultdict(int)
        self.group = defaultdict(list)
        self.max_freq = 0

    def push(self, val):
        self.freq[val] += 1
        f = self.freq[val]
        self.max_freq = max(self.max_freq, f)
        self.group[f].append(val)

    def pop(self):
        val = self.group[self.max_freq].pop()
        self.freq[val] -= 1
        if not self.group[self.max_freq]:
            self.max_freq -= 1
        return val
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

`push(5,7,5,7,4,5):`

push5: freq={5:1},group={1:[5]},max=1.

push7: freq={5:1,7:1},group={1:[5,7]},max=1.

push5: freq={5:2},group={1:[5,7],2:[5]},max=2.

push7: freq={7:2},group={2:[5,7]},max=2.

push4: freq={4:1},group={1:[5,7,4]},max=2.

push5: freq={5:3},group={3:[5]},max=3.

`pop(): group[3]=[5]->pop5. freq[5]=2.`
`group[3]=[]->max=2. return 5 ok`

`pop(): group[2]=[5,7]->pop7. freq[7]=1. return 7 ok`

`pop(): group[2]=[5]->pop5. freq[5]=1.`
`group[2]=[]->max=1. return 5 ok`

Group stacks maintain LRU order within each frequency level automatically

PHASE 6 COMPLEXITY & FOLLOW-UP

push and pop both O(1). Space O(n) across both dicts.

max_freq only decreases when group[max_freq] drains — never increases on pop.

Follow-up: peek without removing — group[max_freq][-1].

Follow-up: if we need O(log n) guarantee — use a sorted dict keyed by (freq, insertion_time).

IN YOUR APP | Expose | Editorial | Community

HARD | Stack | LeetCode #1944 | 12 of 14 | 12/14

Number of Visible People in a Queue

QUESTION

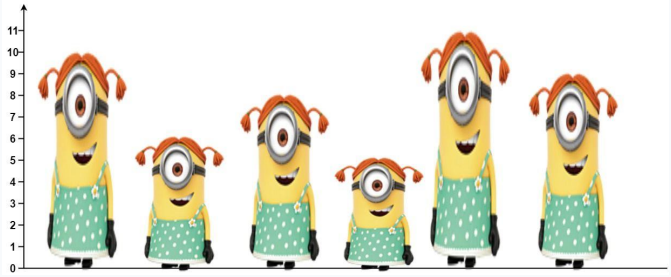
There are n people standing in a queue, and they numbered from 0 to n - 1 in left to right order. You are given an array heights of distinct integers where heights[i] represents the height of the ith person.

A person can see another person to their right in the queue if everybody in between is shorter than both of them. More formally, the ith person can see the jth person if i < j and min(heights[i], heights[j]) > max(heights[i+1], heights[i+2], ..., heights[j-1]).

Return an array answer of length n where answer[i] is the number of people the ith person can see to their right in the queue.

Example 1

Input	`heights = [10,6,8,5,11,9]`
Output	`[3,1,2,1,1,0]`
Explanation	*Person 0 can see person 1, 2, and 4.* *Person 1 can see person 2.* *Person 2 can see person 3 and 4.* *Person 3 can see person 4.* *Person 4 can see person 5.* *Person 5 can see no one since nobody is to the right of them.*



Example 2

Input	`heights = [5,1,2,3,10]`
Output	`[4,1,1,1,0]`

CONSTRAINTS

- n == heights.length
- 1 <= n <= 10^5
- 1 <= heights[i] <= 10^5
- All the values of heights are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

People stand in a queue facing right. Person i sees person j if all people between them are shorter than both.

Count how many people each person can see (to the right). Right?"

"A few quick questions:

Return answer[i] = number of people i can see to the right. Answer includes person j if heights[j] >= heights[i] and all between are shorter."

"Let me walk: heights=[10,6,8,5,11,9] -> [3,1,2,1,1,0] ok"

What should happen on underflow (pop/peek on empty)? Any capacity constraints?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in is:
    walk right keeping the tallest seen so far,
        counting each person taller than all
        before them until one at least as tall as
        person i blocks
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Rewalks the line for every person. Process right to left with a monotonic stack of decreasing heights: the people popped (plus one more if the stack is not empty) are exactly those visible: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack: pop shorter people who can't be seen — O(n). Walk-through:"

Process people right-to-left. stack = []. result = [0]*n.

For i from n-1 to 0:

`count = 0.`

While stack and stack[-1] < heights[i]:

`stack.pop(); count++. <- this person is shorter -> person i can see them`

If stack: count++. <- the next taller person (stack top) blocks further view

`result[i] = count.`

`stack.append(heights[i]).`

Return result.

WHY pop shorter: person i can see all consecutive shorter people until blocked by a taller one.

WHY +1 if stack non-empty: the person who blocked (first taller to the right) is also visible.

PHASE 3 OPTIMIZE

WHY right-to-left: each person looks to the RIGHT -> process their right side first.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1944_NumberOfVisiblePeopleInQueue:
    def canSeePersonsCount(self, heights):
        n = len(heights)
        result = [0] * n
        stack = []
        for i in range(n - 1, -1, -1):
            count = 0
            while stack and heights[i] > stack[-1]:
                stack.pop()
                count += 1
            if stack:
                count += 1
            result[i] = count
            stack.append(heights[i])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

heights=[10,6,8,5,11,9]: process right to left.

i=5(9): stack empty. count=0. stack=[9]. result[5]=0.

i=4(11): 11>9->pop,count=1. stack empty. result[4]=1. stack=[11].

i=3(5): 5<11. count=1. result[3]=1. stack=[11,5].

i=2(8): 8>5->pop,count=1. 8<11->count=2. result[2]=2. stack=[11,8].

i=1(6): 6<8. count=1. result[1]=1. stack=[11,8,6].

i=0(10): 10>6->pop,10>8->pop,count=2. 10<11->count=3. result[0]=3. stack=[11,10]. -> [3,1,2,1,1,0] ok

Each person 'sees' all shorter people popped + 1 taller person blocking the view

Input is a queue of people (heights). Process right-to-left with monotonic stack. deque=[10,6,8,5,11,9] (original queue order). Stack-based solution traverses this queue from right ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
each element pushed/popped once	stack

Follow-up: count visible people in BOTH directions — run the stack twice.

Follow-up: Buildings With an Ocean View (#1762) — similar visibility problem.

IN YOUR APP | Expose | Community

HARD | Stack | LeetCode #2296 | 13 of 14 | 13/14

Design a Text Editor

QUESTION

Design a text editor with a cursor that can do the following:

CONSTRAINTS

- Add text to where the cursor is.
- Delete text from where the cursor is (simulating the backspace key).
- Move the cursor either left or right.

When deleting text, only characters to the left of the cursor will be deleted. The cursor will also remain within the actual text and cannot be moved beyond it. More formally, we have that 0 <= cursor.position <= currentText.length always holds.

Implement the TextEditor class:

- TextEditor() Initializes the object with empty text.
- void addText(string text) Appends text to where the cursor is. The cursor ends to the right of text.
- int deleteText(int k) Deletes k characters to the left of the cursor. Returns the number of characters actually deleted.
- string cursorLeft(int k) Moves the cursor to the left k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.
- string cursorRight(int k) Moves the cursor to the right k times. Returns the last min(10, len) characters to the left of the cursor, where len is the number of characters to the left of the cursor.
- 1 <= text.length, k <= 40
- text consists of lowercase English letters.
- At most 2 * 10^4 calls in total will be made to addText, deleteText, cursorLeft and cursorRight.
- Follow-up: Could you find a solution with time complexity of O(k) per call?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Text editor with addText, deleteText, cursorLeft, cursorRight operations.

cursorLeft/right return the last 10 characters to the left of cursor. Right?"

"A few quick questions:

Cursor moves within the text; delete removes characters to the left of cursor? Yes."

"Let me walk: addText('leetcode'), deleteText(4), addText('practice'), cursorLeft(3), cursorRight(2) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep one string and a cursor
addText / deleteText: rebuild the string around the
    cursor
cursorLeft / cursorRight: move the index and return
    the last 10 characters
```

TIME	SPACE
O(L) per operation	O(L)

WHY IT IS SLOW

Rebuilding the string is O(L) per edit. Two stacks (characters left of the cursor and right of it) make an edit or a cursor move O(k) for k characters moved.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two stacks (left/right of cursor) — O(1) amortized per operation. Walk-through:"

left = stack of chars to the left of cursor. right = stack of chars to right.

addText(text): push each char to left.

deleteText(k): pop min(k, len(left)) chars from left.

cursorLeft(k): move min(k,len(left)) chars from left to right. Return last 10 of left.

cursorRight(k): move min(k,len(right)) chars from right to left. Return last 10 of left.

WHY two stacks: cursor position = boundary between left and right stacks.

Moving cursor = transferring chars between stacks. Efficient for arbitrary cursor movement.

Time: O(k) per move (k = steps). Space: O(total text).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _2296_DesignATextEditor:
    def __init__(self):
        self._left = []
        self._right = []

    def addText(self, text):
        self._left.extend(text)

    def deleteText(self, k):
        deleted = min(k, len(self._left))
        del self._left[-deleted:]
        return deleted

    def cursorLeft(self, k):
        for _ in range(min(k, len(self._left))):
            self._right.append(self._left.pop())
        return ''.join(self._left[-10:])
```

```
    def cursorRight(self, k):
        for _ in range(min(k, len(self._right))):
            self._left.append(self._right.pop())
        return ''.join(self._left[-10:])
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

addText('leetcode'): left=['l','e','e','t','c','o','d','e'].

deleteText(4): removes 'e','d','o','c'. left=['l','e','e','t']. return 4.

addText('practice'): left=['l','e','e','t','p','r','a','c','t','i','c','e'].

cursorLeft(3): move 'e','c','i' to right. left ends in ['t','p','r','a','c','t']. return 'leetpract'.

Left stack = text before cursor; right stack = text after cursor in reverse

Final answer confirmed ok

Two-stack editor: left_stack=['l','e','e','t','p','r','a','c','t','i','c','e'], right_stack=[]. cursorLeft(3): move 'e','c','i' to right. left=['l','e','e','t','p','r','a','c','t']. Last 10='leetpract'+'i'->result='leetpracti' ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k) per cursor, O(n) for add, O(1) for delete	**O(n)**

Follow-up: if cursor jumps are frequent and large, use a rope data structure O(log n).

Follow-up: support undo — maintain a history stack of (operation, parameters).

IN YOUR APP | Expose | Community

HARD | Stack | LeetCode #2444 | 14 of 14 | 14/14

Count Subarrays With Fixed Bounds

QUESTION

You are given an integer array nums and two integers minK and maxK.

A fixed-bound subarray of nums is a subarray that satisfies the following conditions:

CONSTRAINTS

- The minimum value in the subarray is equal to minK.
- The maximum value in the subarray is equal to maxK.

Return the number of fixed-bound subarrays.

A subarray is a contiguous part of an array.

Example 1	
Input	`nums = [1,3,5,2,7,5], minK = 1, maxK = 5`
Output	`2`
Explanation	*The fixed-bound subarrays are [1,3,5] and [1,3,5,2].*

Example 2	
Input	`nums = [1,1,1,1], minK = 1, maxK = 1`
Output	`10`
Explanation	*Every subarray of nums is a fixed-bound subarray. There are 10 possible subarrays.*

- 2 <= nums.length <= 10^5
- 1 <= nums[i], minK, maxK <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count subarrays where min==minK and max==maxK simultaneously. Right?"

"A few quick questions:

Subarray must contain both minK and maxK; no element outside [minK,maxK]? Yes."

"Let me walk: nums=[1,3,5,2,7,5], minK=1, maxK=5 -> ? Let me count.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for i in range(n):
    extend j while all values stay in [minK,
        maxK], tracking the running min and max
    count the subarrays whose min is minK and max
        is maxK
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Restarts from every position. One pass remembering the last position of minK, the last position of maxK and the last out-of-range position: each index adds max(0, min(lastMin, lastMax) - lastBad): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n²) enumeration.

Single pass: track last positions of minK, maxK, and any invalid (out-of-range) element.

For each right end j, count valid subarrays ending at j:

left bound = max(last_invalid+1, 0). Valid if both minK and maxK appeared.

Count += max(0, min(last_min, last_max) - last_invalid).

Time: O(n). Space: O(1)."

Key insight: valid subarrays ending at j = max(0, min(last_minK, last_maxK) - last_invalid) — three running indices computed in one rightward pass give O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2444_CountSubarraysFixedBounds:
    def countSubarrays(self, nums, minK, maxK):
        count = 0
        last_min = last_max = last_invalid = -1
        for j, num in enumerate(nums):
            if num < minK or num > maxK:
                last_invalid = j
            if num == minK:
                last_min = j
            if num == maxK:
                last_max = j
            count += max(0, min(last_min, last_max) - last
_invalid)
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

nums=[1,3,5,2,7,5], minK=1, maxK=5:

j=0(1): last_min=0. min(0,-1)=-1>last_invalid=-1? 0 contribution.

PHASE 5 TEST & DEBUG

j=1(3): no update. j=2(5): last_max=2. min(0,2)=0. 0-(-1)=1. count=1.

j=3(2): no update. min(0,2)=0. 0-(-1)=1. count=2.

j=4(7): 7>maxK=5 -> last_invalid=4. min(0,2)=0. 0-4=-4<0 -> 0. count=2.

j=5(5): last_max=5. min(0,5)=0. 0-4=-4<0 -> 0. count=2. -> 2 ok

Min(last_min,last_max) gives the leftmost required position; subtract last_invalid

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** three index variables

Follow-up: Subarrays with K Different Integers (#992) — at-most-k trick.

Follow-up: if bounds could overlap (minK==maxK), same formula handles it correctly.